SQL Code

```
-- Databricks notebook source
-- DBTITLE 1,Introduction
-- MAGIC %md
-- MAGIC # SQL Data Cleaning & Transformation Pipeline
-- MAGIC
-- MAGIC ## Overview
-- MAGIC This notebook demonstrates a complete data cleaning and transformation workflow:
-- MAGIC
-- MAGIC 1. **Data Cleaning**: Remove nulls, duplicates, and invalid values
-- MAGIC 2. **Daily Sales Analysis**: Aggregate sales by date
-- MAGIC 3. **City-wise Revenue**: Revenue breakdown by location
-- MAGIC 4. **Top Customers**: Identify highest-value customers
-- MAGIC 5. **Repeat Customer Analysis**: Find customers with multiple orders
-- MAGIC 6. **Customer Segmentation**: Classify customers into Gold/Silver/Bronze tiers
-- MAGIC 7. **Final Reporting**: Combine all insights into a comprehensive report
-- MAGIC 8. **Save Output**: Export results for downstream use
-- MAGIC
-- MAGIC **Key Learning Goals:**
-- MAGIC - Understand why data cleaning comes first
-- MAGIC - Practice joins and aggregations
-- MAGIC - Apply business logic for segmentation
-- MAGIC - Build production-ready data pipelines

-- COMMAND ----------

-- DBTITLE 1,Data Cleaning Concepts
-- MAGIC %md
-- MAGIC ## What is Data Cleaning?
-- MAGIC
-- MAGIC Data cleaning is the process of identifying and correcting errors, inconsistencies, and missing values in datasets to ensure data quality before analysis.
-- MAGIC
-- MAGIC ### Key Data Cleaning Steps:
-- MAGIC
-- MAGIC 1. **Remove rows with null keys**: Records without essential identifiers (like customer_id) cannot be joined or analyzed properly
-- MAGIC 2. **Remove duplicate rows**: Duplicates skew counts and aggregations, leading to incorrect metrics
-- MAGIC 3. **Filter invalid values**: Negative amounts, impossible dates, or out-of-range values indicate data errors
-- MAGIC 4. **Check column types**: Ensure data types match expected formats (dates as dates, numbers as numbers)
-- MAGIC 5. **Standardize formats**: Consistent casing, trimming whitespace, standardizing codes
-- MAGIC
-- MAGIC ### Why Clean Data First?
-- MAGIC
```

```sql
-- MAGIC - **Prevents join failures**: Null keys cause LEFT/INNER joins to drop records unexpectedly
-- MAGIC - **Ensures accurate counts**: Duplicates inflate metrics like order counts and revenue
-- MAGIC - **Avoids calculation errors**: Invalid values (negative prices) corrupt aggregations
-- MAGIC - **Improves performance**: Smaller, cleaner datasets process faster
-- MAGIC - **Builds trust**: Clean data produces reliable insights

-- COMMAND ----------

-- DBTITLE 1,Inspect raw customer data
-- First, let's inspect the raw customer data to understand what we're working with
SELECT *
FROM samples.tpch.customer
LIMIT 10;

-- COMMAND ----------

-- DBTITLE 1,Inspect raw orders data
-- Inspect orders data
SELECT *
FROM samples.tpch.orders
LIMIT 10;

-- COMMAND ----------

-- DBTITLE 1,Data quality checks
-- Check data quality issues before cleaning
SELECT
  'Customers' as table_name,
  COUNT(*) as total_rows,
  COUNT(DISTINCT c_custkey) as unique_customers,
  SUM(CASE WHEN c_custkey IS NULL THEN 1 ELSE 0 END) as null_keys,
  SUM(CASE WHEN c_name IS NULL THEN 1 ELSE 0 END) as null_names,
  SUM(CASE WHEN c_acctbal < 0 THEN 1 ELSE 0 END) as negative_balances
FROM samples.tpch.customer

UNION ALL

SELECT
  'Orders' as table_name,
  COUNT(*) as total_rows,
  COUNT(DISTINCT o_orderkey) as unique_orders,
  SUM(CASE WHEN o_custkey IS NULL THEN 1 ELSE 0 END) as null_customer_keys,
  SUM(CASE WHEN o_orderdate IS NULL THEN 1 ELSE 0 END) as null_dates,
  SUM(CASE WHEN o_totalprice < 0 THEN 1 ELSE 0 END) as negative_prices
FROM samples.tpch.orders;

-- COMMAND ----------

-- DBTITLE 1,Task 1 Overview
-- MAGIC %md
-- MAGIC ## Task 1: Daily Sales Analysis
```

```
-- MAGIC
-- MAGIC **Objective**: Aggregate total sales and order counts by date to identify daily revenue trends.
-- MAGIC
-- MAGIC **Output**: `date`, `total_sales`, `order_count`
-- MAGIC
-- MAGIC **Why this matters**: Daily sales trends help identify peak periods, seasonal patterns, and revenue fluctuations.

-- COMMAND ----------

-- DBTITLE 1,Task 2 Overview
-- MAGIC %md
-- MAGIC ## Task 2: City-wise Revenue Analysis
-- MAGIC
-- MAGIC **Objective**: Calculate total revenue, customer count, and order count for each city.
-- MAGIC
-- MAGIC **Output**: `city`, `total_revenue`, `customer_count`, `order_count`
-- MAGIC
-- MAGIC **Why this matters**: Geographic analysis helps identify high-performing markets and guides regional strategy.

-- COMMAND ----------

-- DBTITLE 1,Task 3 Overview
-- MAGIC %md
-- MAGIC ## Task 3: Top 5 Customers
-- MAGIC
-- MAGIC **Objective**: Identify the highest-spending customers to focus retention and VIP programs.
-- MAGIC
-- MAGIC **Output**: `customer_name`, `customer_id`, `total_spend`, `order_count`
-- MAGIC
-- MAGIC **Why this matters**: Top customers often represent a disproportionate share of revenue (Pareto principle).

-- COMMAND ----------

-- DBTITLE 1,Task 4 Overview
-- MAGIC %md
-- MAGIC ## Task 4: Repeat Customer Analysis
-- MAGIC
-- MAGIC **Objective**: Find customers with more than one order to measure customer loyalty.
-- MAGIC
-- MAGIC **Output**: `customer_id`, `order_count`, `total_spend`
-- MAGIC
-- MAGIC **Why this matters**: Repeat customers indicate satisfaction and are cheaper to retain than acquiring new ones.

-- COMMAND ----------

-- DBTITLE 1,Task 5 Overview
```

```sql
-- MAGIC %md
-- MAGIC ## Task 5: Customer Segmentation
-- MAGIC
-- MAGIC **Objective**: Classify customers into Gold/Silver/Bronze tiers based on total spending.
-- MAGIC
-- MAGIC **Business Rules**:
-- MAGIC - **Gold**: `total_spend > 10,000` - Premium customers
-- MAGIC - **Silver**: `total_spend 5,000-10,000` - Mid-tier customers
-- MAGIC - **Bronze**: `total_spend < 5,000` - Standard customers
-- MAGIC
-- MAGIC **Output**: `customer_name`, `customer_id`, `total_spend`, `segment`
-- MAGIC
-- MAGIC **Why this matters**: Segmentation enables targeted marketing, personalized service levels, and resource prioritization.

-- COMMAND ----------

-- DBTITLE 1,Task 6 Overview
-- MAGIC %md
-- MAGIC ## Task 6: Final Reporting Table
-- MAGIC
-- MAGIC **Objective**: Combine all insights into a comprehensive customer report for business stakeholders.
-- MAGIC
-- MAGIC **Output**: `customer_name`, `city`, `total_spend`, `order_count`, `segment`, `first_order_date`, `last_order_date`
-- MAGIC
-- MAGIC **Why this matters**: A unified view enables holistic customer understanding and supports strategic decision-making.

-- COMMAND ----------

-- DBTITLE 1,Clean customer data
-- Step 1: Clean customer data
-- Remove nulls, duplicates, and ensure data quality
CREATE OR REPLACE TEMP VIEW clean_customers AS
SELECT DISTINCT
  c_custkey as customer_id,
  c_name as customer_name,
  SPLIT(c_address, ',')[0] as city,  -- Extract city from address
  c_acctbal as account_balance
FROM samples.tpch.customer
WHERE c_custkey IS NOT NULL  -- Remove null keys
  AND c_name IS NOT NULL     -- Remove null names
  AND c_acctbal >= 0;        -- Filter invalid balances

SELECT COUNT(*) as cleaned_customer_count FROM clean_customers;

-- COMMAND ----------

-- DBTITLE 1,Clean orders data
-- Step 2: Clean orders data
CREATE OR REPLACE TEMP VIEW clean_orders AS
```

```sql
SELECT DISTINCT
  o_orderkey as order_id,
  o_custkey as customer_id,
  o_orderdate as order_date,
  o_totalprice as total_price
FROM samples.tpch.orders
WHERE o_custkey IS NOT NULL      -- Remove null customer keys
  AND o_orderkey IS NOT NULL     -- Remove null order keys
  AND o_orderdate IS NOT NULL    -- Remove null dates
  AND o_totalprice > 0;          -- Filter invalid prices (must be positive)

SELECT COUNT(*) as cleaned_order_count FROM clean_orders;

-- COMMAND ----------

-- DBTITLE 1,Task 1: Daily Sales
-- Task 1: Daily Sales → Output: date, total_sales
CREATE OR REPLACE TEMP VIEW daily_sales AS
SELECT
  order_date as date,
  ROUND(SUM(total_price), 2) as total_sales,
  COUNT(order_id) as order_count
FROM clean_orders
GROUP BY order_date
ORDER BY date DESC
LIMIT 30;  -- Show last 30 days

SELECT * FROM daily_sales;

-- COMMAND ----------

-- DBTITLE 1,Task 2: City-wise Revenue
-- Task 2: City-wise Revenue → Output: city, total_revenue
CREATE OR REPLACE TEMP VIEW city_revenue AS
SELECT
  c.city,
  ROUND(SUM(o.total_price), 2) as total_revenue,
  COUNT(DISTINCT o.customer_id) as customer_count,
  COUNT(o.order_id) as order_count
FROM clean_orders o
INNER JOIN clean_customers c ON o.customer_id = c.customer_id
GROUP BY c.city
ORDER BY total_revenue DESC
LIMIT 20;

SELECT * FROM city_revenue;

-- COMMAND ----------

-- DBTITLE 1,Task 3: Top 5 Customers
-- Task 3: Top 5 Customers → Output: customer_name, total_spend
CREATE OR REPLACE TEMP VIEW top_customers AS
SELECT
  c.customer_name,
```

```sql
  c.customer_id,
  ROUND(SUM(o.total_price), 2) as total_spend,
  COUNT(o.order_id) as order_count
FROM clean_orders o
INNER JOIN clean_customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_spend DESC
LIMIT 5;

SELECT * FROM top_customers;

-- COMMAND ----------

-- DBTITLE 1,Task 4: Repeat Customers
-- Task 4: Repeat Customers (>1 order) → Output: customer_id, order_count
CREATE OR REPLACE TEMP VIEW repeat_customers AS
SELECT
  customer_id,
  COUNT(order_id) as order_count,
  ROUND(SUM(total_price), 2) as total_spend
FROM clean_orders
GROUP BY customer_id
HAVING COUNT(order_id) > 1  -- Only customers with more than 1 order
ORDER BY order_count DESC
LIMIT 20;

SELECT * FROM repeat_customers;

-- COMMAND ----------

-- DBTITLE 1,Task 5: Customer Segmentation
-- Task 5: Customer Segmentation → Gold/Silver/Bronze
-- Business Logic:
--   total_spend > 10000 → Gold
--   total_spend 5000-10000 → Silver
--   total_spend < 5000 → Bronze

CREATE OR REPLACE TEMP VIEW customer_segments AS
SELECT
  c.customer_name,
  c.customer_id,
  ROUND(SUM(o.total_price), 2) as total_spend,
  CASE
    WHEN SUM(o.total_price) > 10000 THEN 'Gold'
    WHEN SUM(o.total_price) >= 5000 THEN 'Silver'
    ELSE 'Bronze'
  END as segment
FROM clean_orders o
INNER JOIN clean_customers c ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.customer_name;

-- Show segment distribution
SELECT
  segment,
```

```sql
  COUNT(*) as customer_count,
  ROUND(SUM(total_spend), 2) as total_revenue,
  ROUND(AVG(total_spend), 2) as avg_spend_per_customer
FROM customer_segments
GROUP BY segment
ORDER BY
  CASE segment
    WHEN 'Gold' THEN 1
    WHEN 'Silver' THEN 2
    ELSE 3
  END;

-- COMMAND ----------

-- DBTITLE 1,Task 6: Final Reporting Table
-- Task 6: Final Reporting Table
-- Combine all insights: customer_name, city, total_spend, order_count, segment

CREATE OR REPLACE TEMP VIEW final_report AS
SELECT
  c.customer_name,
  c.city,
  seg.total_spend,
  COUNT(o.order_id) as order_count,
  seg.segment,
  MIN(o.order_date) as first_order_date,
  MAX(o.order_date) as last_order_date
FROM clean_customers c
INNER JOIN clean_orders o ON c.customer_id = o.customer_id
INNER JOIN customer_segments seg ON c.customer_id = seg.customer_id
GROUP BY c.customer_id, c.customer_name, c.city, seg.total_spend, seg.segment
ORDER BY seg.total_spend DESC;

-- Display top 20 customers from final report
SELECT * FROM final_report LIMIT 20;

-- COMMAND ----------

-- DBTITLE 1,Report summary statistics
-- Summary statistics for the final report
SELECT
  COUNT(DISTINCT customer_name) as total_customers,
  COUNT(DISTINCT city) as total_cities,
  SUM(order_count) as total_orders,
  ROUND(SUM(total_spend), 2) as total_revenue,
  ROUND(AVG(total_spend), 2) as avg_customer_value,
  ROUND(AVG(order_count), 2) as avg_orders_per_customer
FROM final_report;

-- COMMAND ----------

-- DBTITLE 1,Task 7 Overview
-- MAGIC %md
-- MAGIC ## Task 7: Save Output
```

```
-- MAGIC
-- MAGIC **Objective**: Persist the final report as a permanent Delta table for downstream consumption.
-- MAGIC
-- MAGIC **Output**: Permanent table `workspace.default.customer_report`
-- MAGIC
-- MAGIC **Why this matters**: Temporary views disappear when the session ends. Permanent tables enable:
-- MAGIC - Sharing results across teams
-- MAGIC - Scheduled refreshes and pipelines
-- MAGIC - Historical tracking and versioning
-- MAGIC - Integration with BI tools and dashboards

-- COMMAND ----------

-- DBTITLE 1,Task 7: Save Output
-- Task 7: Save Output → Save final report to a permanent table
-- Create a permanent Delta table from the final report
CREATE OR REPLACE TABLE workspace.default.customer_report AS
SELECT * FROM final_report;

-- Verify the table was created and show row count
SELECT
  'customer_report' as table_name,
  COUNT(*) as total_rows,
  COUNT(DISTINCT customer_name) as unique_customers,
  COUNT(DISTINCT segment) as segments
FROM workspace.default.customer_report;

-- COMMAND ----------

-- DBTITLE 1,Alternative: Export as CSV
-- MAGIC %python
-- MAGIC # Alternative: Export the report as CSV file
-- MAGIC # Read the saved table
-- MAGIC df = spark.table("workspace.default.customer_report")
-- MAGIC
-- MAGIC # Show summary of what will be exported
-- MAGIC print(f"Total records to export: {df.count()}")
-- MAGIC print(f"\nSample of data:")
-- MAGIC display(df.limit(10))
-- MAGIC
-- MAGIC # Note: To export as CSV in production, you would use:
-- MAGIC # output_path = "/Volumes/catalog/schema/volume/customer_report.csv"
-- MAGIC # df.coalesce(1).write.mode('overwrite').option('header', 'true').csv(output_path)
-- MAGIC print("\n✅ Table saved successfully as workspace.default.customer_report")
-- MAGIC print("To export as CSV, configure a Unity Catalog Volume path")

-- COMMAND ----------

-- DBTITLE 1,Reflection Questions
-- MAGIC %md
-- MAGIC ## Reflection Questions (Very Important)
```

```
-- MAGIC
-- MAGIC ### 1. Why is cleaning done before joining tables?
-- MAGIC **Answer**: Cleaning must happen first because:
-- MAGIC - **Null keys cause join failures**: If customer_id is NULL, INNER/LEFT joins won't match records, silently dropping data
-- MAGIC - **Duplicates inflate results**: If orders table has duplicates, revenue totals will be wrong after joining
-- MAGIC - **Invalid values corrupt calculations**: Negative prices or impossible dates create incorrect aggregations
-- MAGIC - **Performance**: Cleaning reduces row count, making subsequent joins faster
-- MAGIC - **Data integrity**: Ensures every record in the final output is valid and trustworthy
-- MAGIC
-- MAGIC ### 2. What would go wrong if null keys are not removed?
-- MAGIC **Answer**:
-- MAGIC - **INNER JOIN**: Records with null keys are silently excluded from results, reducing counts without warning
-- MAGIC - **LEFT JOIN**: NULL keys appear in output but can't match to the other table, creating incomplete records
-- MAGIC - **Aggregations**: GROUP BY treats NULLs as a separate group, skewing segment counts
-- MAGIC - **Business logic**: Downstream applications expecting valid IDs will crash or produce errors
-- MAGIC - **Example**: A customer with NULL customer_id appears in orders but never shows up in city revenue reports
-- MAGIC
-- MAGIC ### 3. How did you decide join order?
-- MAGIC **Answer**: Join order follows the **dimension → fact → aggregation** pattern:
-- MAGIC 1. **Start with cleaned base tables** (clean_customers, clean_orders)
-- MAGIC 2. **Join orders to customers** since orders reference customers (foreign key relationship)
-- MAGIC 3. **Add derived tables last** (customer_segments) after base aggregations are complete
-- MAGIC 4. **Principle**: Join smallest table first when possible, and ensure keys are indexed
-- MAGIC 5. **In this pipeline**: orders (fact) → customers (dimension) → segments (derived)
-- MAGIC
-- MAGIC ### 4. Which step was most difficult and why?
-- MAGIC **Answer**: Typically the **customer segmentation** (Task 5) is most challenging because:
-- MAGIC - **Business logic complexity**: Translating business rules (Gold/Silver/Bronze) into SQL CASE statements
-- MAGIC - **Aggregation before segmentation**: Must calculate total_spend per customer BEFORE applying segment logic
-- MAGIC - **Multiple conditions**: Ensuring threshold boundaries are correct (>10000 vs >=10000)
-- MAGIC - **Validation**: Verifying segments make business sense (Gold customers should have highest spend)
-- MAGIC
-- MAGIC ### 5. How is SQL logic similar to PySpark?
-- MAGIC **Answer**:
-- MAGIC | Concept | SQL | PySpark |
```

```
-- MAGIC |---------|-----|----------|
-- MAGIC | Filtering | `WHERE customer_id IS NOT NULL` | `.filter(col('customer_id').isNotNull())` |
-- MAGIC | Aggregation | `SUM(total_price)` | `.agg(sum('total_price'))` |
-- MAGIC | Grouping | `GROUP BY customer_id` | `.groupBy('customer_id')` |
-- MAGIC | Joins | `INNER JOIN orders ON...` | `.join(orders, on=..., how='inner')` |
-- MAGIC | Case logic | `CASE WHEN ... THEN ... END` | `.when(...).otherwise(...)` |
-- MAGIC
-- MAGIC **Both are declarative** (describe what you want, not how to compute it) and **both use lazy evaluation** (build execution plan, execute on action).
-- MAGIC
-- MAGIC ### 6. What challenges will appear with large data?
-- MAGIC **Answer**:
-- MAGIC - **Memory pressure**: Full table scans and wide joins can exceed cluster memory
-- MAGIC - **Skew**: Uneven data distribution (one customer with millions of orders) slows down specific tasks
-- MAGIC - **Shuffles**: GROUP BY and JOIN operations shuffle data across nodes, creating network bottlenecks
-- MAGIC - **Timeouts**: Complex queries without proper filtering (date ranges) take too long
-- MAGIC - **Duplicate handling**: `SELECT DISTINCT` on billions of rows is expensive
-- MAGIC - **Mitigation**: Partition data by date, use broadcast joins for small dimension tables, add indexes, tune cluster size
-- MAGIC
-- MAGIC ### 7. Can you explain your pipeline in simple steps?
-- MAGIC **Answer**:
-- MAGIC 1. **Load raw data**: Read customers and orders tables
-- MAGIC 2. **Inspect quality**: Check for nulls, duplicates, invalid values
-- MAGIC 3. **Clean each table separately**: Remove bad records, standardize formats
-- MAGIC 4. **Create daily sales**: Aggregate orders by date to see trends
-- MAGIC 5. **Create city revenue**: Join cleaned orders + customers, group by city
-- MAGIC 6. **Find top customers**: Aggregate spend per customer, rank by total
-- MAGIC 7. **Find repeat customers**: Count orders per customer, filter >1
-- MAGIC 8. **Segment customers**: Apply business rules to classify Gold/Silver/Bronze
-- MAGIC 9. **Build final report**: Combine all metrics (city, spend, orders, segment) into one table
-- MAGIC 10. **Validate results**: Check totals, verify business logic, review sample records
-- MAGIC 11. **Save output**: Write to Delta table or file for downstream use
-- MAGIC
-- MAGIC **Key principle**: Clean → Transform → Aggregate → Combine → Validate → Save

-- COMMAND ----------


```